

Comparative Analysis of Deep Learning Algorithms for Image Classification and Clustering: Supervised vs. Unsupervised Learning Approaches on Cars and STL-10 Datasets

Bandar Qaid Ahmed Abdulrab^{*1}, Enggal Meureksa Ibrahim², Vani Oktaviani³, Sinta Nuria⁴

¹ Informatika, Fakultas Sains Teknologi, UIN Saizu, Purbalingga, Indonesia.

² Informatika, Fakultas Sains Teknologi, UIN Saizu, Purbalingga, Indonesia.

Article Information

Submitted mmmdd, yyyy

Accepted mmmdd, yyyy

Published mmmdd, yyyy

Abstract

Image classification is a fundamental task in computer vision that has been revolutionized by deep learning techniques. This study presents a comparative analysis of deep learning algorithms applied to two distinct image recognition paradigms: supervised and unsupervised learning. In the supervised learning experiment, a Custom Convolutional Neural Network (CNN) built from scratch is benchmarked against Transfer Learning using MobileNetV2, applied to a multi-class car brand classification dataset with an 80:20 training-validation split over ten epochs. Performance is measured through accuracy, precision, recall, and F1-score. In the unsupervised learning experiment, traditional K-Means clustering applied to flattened raw pixel vectors is compared against a hybrid Convolutional Autoencoder combined with K-Means, applied to the STL-10 animal image dataset (5,000 images, 10 clusters). Cluster quality is evaluated using Silhouette Score and Davies-Bouldin Index. Experimental results show that Transfer Learning (MobileNetV2) substantially outperforms Custom CNN by leveraging ImageNet pre-trained feature representations, yielding higher accuracy with limited training data and epochs. Similarly, Autoencoder-enhanced K-Means outperforms raw-pixel K-Means by compressing images into compact latent representations that preserve structural and geometric features. These findings consistently confirm the superiority of feature-reuse strategies in both supervised and unsupervised deep learning regimes.

Keywords: Deep Learning; Transfer Learning; Convolutional Neural Network; Autoencoder; Image Clustering.

1. Introduction

Image classification has become one of the most actively researched problems in computer vision and machine learning. The task of assigning a semantic label to an input image lies at the foundation of applications ranging from autonomous vehicles and medical imaging to industrial quality control and surveillance systems [1]. The emergence of deep learning—particularly Convolutional Neural Networks (CNNs)—has profoundly transformed this field by enabling automatic feature extraction directly from raw pixel data, thereby eliminating the need for handcrafted feature engineering [2].

Within the deep learning paradigm, supervised learning has achieved remarkable performance when sufficient labeled training data is available [3]. However, collecting and annotating large-scale image datasets is costly and time-consuming. Transfer Learning addresses this limitation by re-using feature representations learned from large general-purpose datasets such as ImageNet, adapting them to domain-specific tasks with comparatively few labeled examples [4]. On the other hand, unsupervised learning methods, such as clustering, provide an alternative route when labels are completely absent. Traditional clustering algorithms like K-Means, however,

* Author Correspondence: Author name: Affiliation, Address, City, Province – Country. Email: mail@affiliation.ac.id.

suffer from the curse of dimensionality when applied directly to high-dimensional raw image pixels [5]. Deep autoencoders offer a principled way to learn compact, semantically meaningful representations in an unsupervised manner, enabling more effective downstream clustering [6].

Despite the abundance of individual studies on supervised CNN classification and unsupervised image clustering, comparative analyses that systematically evaluate both learning paradigms under controlled experimental conditions on real-world image datasets remain scarce. This study addresses that gap by conducting controlled side-by-side comparisons across two axes: (1) Custom CNN versus Transfer Learning (MobileNetV2) on a multi-class car brand classification task, and (2) K-Means on raw pixels versus Convolutional Autoencoder combined with K-Means on the STL-10 animal image dataset. The objective is to provide practitioners with empirical evidence supporting algorithm selection under different data availability and computational constraints. The remainder of this paper is organized as follows: Section 2 reviews related work; Section 3 describes the datasets, architectures, and evaluation metrics; Section 4 presents experimental results and analysis; Section 5 offers a broader discussion; and Section 6 concludes the study.

2. Related Research

Convolutional Neural Networks were introduced as a breakthrough architecture for image recognition by LeCun et al. [7], and gained widespread recognition following the landmark AlexNet study by Krizhevsky et al. [2], which achieved significantly lower error rates than traditional methods on the ImageNet

benchmark. Subsequent architectures such as VGGNet [8], GoogLeNet [9], and ResNet [10] progressively improved performance by exploring deeper and wider network configurations. These studies collectively established CNN as the de facto standard for supervised image classification tasks.

Transfer Learning has emerged as a dominant strategy when labeled datasets are small or domain-specific. The principle of fine-tuning a network pre-trained on ImageNet was popularized by Razavian et al. [11], who demonstrated that CNN features transfer remarkably well across visual domains. MobileNetV2, proposed by Sandler et al. [4], extended this line of research by introducing inverted residuals and linear bottlenecks, yielding a lightweight yet highly accurate model suitable for resource-constrained environments. Comparative studies have consistently shown that Transfer Learning achieves superior performance over scratch-trained CNNs when the number of labeled examples is limited [12].

In the unsupervised domain, K-Means clustering is among the most widely used partitioning algorithms [13]. However, its performance degrades significantly when applied to raw image pixels due to high dimensionality, noise sensitivity, and the lack of perceptual structure in Euclidean pixel space. Autoencoders, as introduced in their modern form by Vincent et al. [14] and theoretically grounded by Bengio et al. [6], learn compressed latent representations by minimizing reconstruction loss. Deep clustering approaches that combine autoencoders with K-Means have demonstrated consistent improvements over pixel-space clustering on visual benchmarks [15][16]. The STL-10 dataset [17], designed specifically to evaluate unsupervised and semi-

supervised feature learning, has become a standard benchmark in this line of research.

3. Method

This section describes the datasets used, the architecture of each algorithm, the training configuration, and the evaluation metrics employed in both supervised and unsupervised experiments. All implementations were developed in Python 3 in Google Colab using TensorFlow 2 / Keras [18] for neural network components and Scikit-learn [19] for traditional machine learning components. Experiments were executed on Google Colaboratory with GPU acceleration.

3.1 Supervised Learning: Architecture Design

Two model architectures were implemented and trained for ten epochs each under identical data loading and preprocessing conditions to ensure a fair comparison. Both models received the same 128×128 pixel input resolution, the same 80:20 training-validation split, and the same Adam optimizer with sparse categorical cross-entropy loss.

3.1.1 Custom CNN (From Scratch)

The Custom CNN was constructed as a Sequential stack, initializing all weights randomly prior to training. The network consists of three successive convolutional blocks, each pairing a Conv2D layer with a MaxPooling2D layer to progressively extract spatial features while reducing spatial dimensionality. Filter depths increase from 32 to 64 to 128 across the three blocks, following the standard pyramid design principle wherein early layers detect low-level features (edges, textures) and deeper layers combine them into higher-level representations. A Dropout layer with a rate of 0.5 was applied

before the output layer to mitigate overfitting. The complete layer configuration and output shapes are shown in the following table.

Rescaling	Scale: 1/255	$128 \times 128 \times 3$
Conv2D (1)	32 filters, 3×3 , ReLU	$126 \times 126 \times 32$
MaxPooling2D (1)	Pool: 2×2	$63 \times 63 \times 32$
Conv2D (2)	64 filters, 3×3 , ReLU	$61 \times 61 \times 64$
MaxPooling2D (2)	Pool: 2×2	$30 \times 30 \times 64$
Conv2D (3)	128 filters, 3×3 , ReLU	$28 \times 28 \times 128$
MaxPooling2D (3)	Pool: 2×2	$14 \times 14 \times 128$
Flatten	—	25,088
Dense	128 units, ReLU	128
Dropout	Rate: 0.5	128
Dense (Output)	N classes, Softmax	N

Table 1. Custom CNN Architecture Layer Configuration

3.1.2 Transfer Learning — MobileNetV2

The Transfer Learning model employs MobileNetV2, a lightweight depthwise separable convolutional network pre-trained on the ImageNet dataset (1,000 classes, 1.2 million images). The base model weights were kept fully frozen (trainable = False) throughout the ten training epochs, meaning only the classification head appended on top was updated via gradient descent. This approach leverages the rich, generalisable visual feature hierarchy already encoded in MobileNetV2's weights — ranging from edge detectors in early layers to object-part detectors in deeper layers — without the computational and data cost of learning these representations from scratch. The input rescaling range was set to $[-1, 1]$ as required by

MobileNetV2's original preprocessing specification, differing from the $[0, 1]$ normalisation used in the Custom CNN. A GlobalAveragePooling2D layer was used instead of Flatten to reduce spatial feature maps to a single vector per channel, which reduces the parameter count and acts as an implicit regulariser. The full head configuration is shown in the following table.

Rescaling	Scale: 1/127.5, offset: -1	MobileNet V2 input range $[-1, 1]$
MobileNetV2 Base	Frozen, ImageNet weights	1,280 feature channels
GlobalAveragePooling2D	—	Spatial average
Dense	128 units, ReLU	Trainable head
Dropout	Rate: 0.2	Regularization
Dense (Output)	N classes, Softmax	Classification

Table 2. Transfer Learning (MobileNetV2) Architecture Configuration

3.2 Unsupervised Learning: Architecture Design

3.2.1 K-Means on Raw Pixels

As a baseline, K-Means was applied directly to the raw pixel representation of each image. Each $96 \times 96 \times 3$ image was flattened into a one-dimensional vector of 27,648 dimensions, with pixel values normalized to the $[0, 1]$ range. The K-Means algorithm was configured with $K = 10$ clusters to match the ten ground-truth categories in the STL-10 dataset, k-means++ initialization to improve centroid seed selection, ten independent restarts ($n_{\text{init}} = 10$) to reduce sensitivity to initialization, and a fixed random seed of 42 for reproducibility. No dimensionality

reduction was applied prior to clustering, making this condition a direct measure of K-Means performance in native pixel space.

3.2.2 Autoencoder + K-Means

A Convolutional Autoencoder was designed and trained in a fully unsupervised fashion, using each image as both input and reconstruction target. The network is composed of two symmetric sub-networks: an encoder that progressively compresses the input image into a compact latent representation, and a decoder that reconstructs the original image from that latent code. Training minimizes mean squared error (MSE) between the original and reconstructed images, forcing the encoder to retain only the most structurally informative features. After training, the decoder is discarded and only the encoder sub-model is retained for feature extraction. The latent feature vector for each image is then flattened and passed to K-Means (same configuration as Section 3.3.1) for clustering. The complete encoder-decoder architecture is detailed in the following table.

Encoder	Conv2D	32 filters, 3×3 , ReLU, same padding	$96 \times 96 \times 32$
Encoder	MaxPooling2D	2×2	$48 \times 48 \times 32$
Encoder	Conv2D	64 filters, 3×3 , ReLU	$48 \times 48 \times 64$
Encoder	MaxPooling2D	2×2	$24 \times 24 \times 64$
Encoder (Latent)	Conv2D	128 filters, 3×3 , ReLU	$24 \times 24 \times 128$
			←

Decoder	Conv2D	128 filters, 3×3, ReLU	Latent Space 24 × 24 × 128
Decoder	UpSampling 2D	2×2	48 × 48 × 128
Decoder	Conv2D	64 filters, 3×3, ReLU	48 × 48 × 64
Decoder	UpSampling 2D	2×2	96 × 96 × 64
Decoder (Output)	Conv2D	3 filters, 3×3, Sigmoid	96 × 96 × 3

Table 3. Convolutional Autoencoder Architecture

3.3 Evaluation Metrics

3.3.1 Supervised Metrics

Model performance in the supervised experiment was evaluated using four standard classification metrics computed on the validation set via weighted averaging across all classes. Weighted averaging was chosen over macro averaging because it accounts for class imbalance by weighting each class's contribution proportionally to its support in the validation set.

$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$	(1)
$\text{Precision} = \frac{TP}{TP + FP}$	(2)
$\text{Recall} = \frac{TP}{TP + FN}$	(3)
$\text{F1-Score} = \frac{2 \times (\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})}$	(4)

Where TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives, respectively. Accuracy measures the overall fraction of correctly classified samples.

Precision quantifies how many positively predicted samples were truly positive. Recall measures how many actual positives were correctly retrieved. F1-Score is the harmonic mean of Precision and Recall, providing a balanced metric that penalises models that perform well on only one of the two.

3.3.2 Unsupervised Metrics

Since ground-truth labels are not used during clustering, performance is assessed using two internal cluster validity indices that rely solely on the data's geometric structure.

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \quad s(i) \in [-1, 1] \quad (5)$$

The Silhouette Score $s(i)$ measures how similar sample i is to its assigned cluster relative to the nearest alternative cluster, where $a(i)$ is the mean intra-cluster distance and $b(i)$ is the mean distance to the nearest neighbouring cluster. Values range from -1 to 1 ; values close to 1 indicate that samples are well-matched to their own cluster and clearly separated from others, while values near 0 suggest overlapping clusters.

$$DB = \frac{1}{k} \times \sum_i \max_{j \neq i} \left[\frac{(\sigma_i + \sigma_j)}{d(c_i, c_j)} \right] \quad (6)$$

The Davies-Bouldin Index (DB) measures the average ratio of within-cluster scatter to between-cluster separation across all cluster pairs, where σ_i is the average distance of all points in cluster i from centroid c_i , and $d(c_i, c_j)$ is the distance between centroids i and j . Lower values indicate better-defined, more separated clusters. Unlike the Silhouette Score, DB directly penalises clusters that are simultaneously wide and close together.

3.4 Training Procedure

3.4.1 Custom CNN Training

The training procedure for the Custom CNN follows a standard supervised mini-batch gradient descent loop. Weights are randomly initialised and updated over ten epochs using the Adam optimiser. At the end of each epoch, the model is evaluated on the validation set to monitor generalisation. The full procedure is formalised in Algorithm 1.

Algorithm 1: Custom CNN Training (Supervised)

Input: Training dataset D_{train} ,
Validation dataset D_{val}
Output: Trained model M_{cnn} ,
Evaluation metrics

```

1: Initialize CNN weights
   randomly (Xavier initialization)
2: Configure optimizer: Adam (lr
   = 0.001)
3: Configure loss: Sparse
   Categorical Cross-Entropy
4: FOR epoch = 1 TO 10 DO
5:   FOR each mini-batch  $b \in$ 
    $D_{train}$  (batch_size = 32) DO
6:     Forward pass:  $\hat{y} =$ 
    $CNN(x_b)$ 
7:     Compute loss:  $L = -\sum$ 
    $y \cdot \log(\hat{y})$ 
8:     Backward pass:  $\partial L / \partial W$ 
   for all layers
9:     Update weights:  $W \leftarrow W$ 
    $- lr \cdot \nabla W$ 
10:   END FOR
11:   Evaluate on  $D_{val}$ : compute
   val_accuracy, val_loss
12: END FOR
13: Compute  $y_{pred} =$ 
    $\text{argmax}(CNN(D_{val}))$ 
14: RETURN Accuracy, Precision,
   Recall, F1-Score, Confusion Matrix

```

3.4.2 Autoencoder + K-Means Training

The unsupervised training procedure is divided into two sequential phases. In Phase 1, the full autoencoder (encoder + decoder) is trained to minimise image reconstruction loss, learning a compressed feature space without any label supervision. In Phase 2, the trained encoder is used to extract latent feature vectors from all images, which are then clustered using K-Means. The two-phase procedure is formalised in Algorithm 2.

Algorithm 2: Autoencoder + K-Means Clustering (Unsupervised)

Input: Unlabeled image dataset X
($N=5000$ images, $96 \times 96 \times 3$)
Output: Cluster assignments C ,
Evaluation metrics

```

Phase 1: Autoencoder Feature
Learning
1: Initialize Encoder  $E$  and
   Decoder  $D$  (random weights)
2: FOR epoch = 1 TO 5 DO
3:   FOR each mini-batch  $b \in X$ 
   (batch_size = 64) DO
4:      $Z = E(x_b)$  //
   Encode to latent space
5:      $\hat{X} = D(Z)$ 
   // Reconstruct image
6:      $L = \text{MSE}(x_b, \hat{X})$  //
   Reconstruction loss
7:   Update  $E, D$  via Adam
   optimizer
8:   END FOR
9: END FOR

Phase 2: Feature Extraction +
Clustering
10: FOR each image  $x_i \in X$  DO
11:    $z_i = E(x_i)$  // Extract
   latent representation
12: END FOR

```

```

13: Z_flat = flatten(Z) // Shape:
    (5000, D_latent)
14: Initialize K-Means: K=10,
    n_init=10, seed=42
15: C = KMeans.fit_predict(Z_flat)
16: Compute Silhouette Score s(i)
    for all i ∈ X
17: Compute Davies-Bouldin Index
    DB
18: RETURN C, s- (mean Silhouette
    Score), DB

```

3.6 Computer Code

All experiments were implemented in Python 3 in Google Colab using TensorFlow 2 / Keras for neural network construction and training, and Scikit-learn for traditional machine learning components and evaluation metrics. The complete implementation is structured across two Jupyter Notebook files: one for the supervised classification experiment and one for the unsupervised clustering experiment. Key code excerpts are presented below to support reproducibility.

3.6.1 Data Loading and Preprocessing — Supervised (Cars Dataset)

The Cars Dataset was loaded using TensorFlow's `image_dataset_from_directory` utility, which handles directory-based label inference automatically. Images were resized to 128×128 pixels and organised into training and validation subsets with an 80:20 split and a fixed seed of 123 to ensure deterministic partitioning across runs. The `shuffle=False` flag was applied exclusively to the validation subset so that the ordering of ground-truth labels remained consistent with the model's prediction output during metric computation.

```
!pip install opendatasets
```

```

import opendatasets as od
import tensorflow as tf
from tensorflow import keras

from keras import layers, models

from sklearn.metrics import
accuracy_score, precision_score,
recall_score, f1_score,
confusion_matrix

import matplotlib.pyplot as plt

import seaborn as sns

import numpy as np

import os

# Download Cars Dataset dari Kaggle
print("Downloading Cars Dataset...")

od.download('https://www.kaggle.com/datasets/kshitij192/cars-image-dataset')

# Menentukan direktori dataset (kita
gunakan folder train untuk di-split)

data_dir = './cars-image-dataset/Cars
Dataset/train'

print("\nSetup & Download Selesai!")

IMG_SIZE = 128

BATCH_SIZE = 32

EPOCHS = 10 # Kita gunakan 10 epoch
untuk perbandingan yang adil

# 1. Memuat Dataset dan membaginya
menjadi Train & Validation (80:20)

print("Memuat Data Training...")

```

```

train_ds =
tf.keras.utils.image_dataset_from_directory(
    data_dir, validation_split=0.2,
    subset="training", seed=123,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE
)

print("\nMemuat Data Validasi...")
# PENTING: shuffle=False agar urutan
label asli (y_true) sama dengan hasil
prediksi

val_ds =
tf.keras.utils.image_dataset_from_directory(
    data_dir, validation_split=0.2,
    subset="validation", seed=123,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE, shuffle=False
)

class_names = train_ds.class_names
num_classes = len(class_names)

print(f"\nJumlah Kelas: {num_classes}
-> {class_names}")

# Mengambil label asli (y_true) dari
validation dataset untuk perhitungan
metrics

y_true = np.concatenate([y for x, y in
val_ds], axis=0)

# Fungsi untuk menghitung dan mencetak
Metrics (Accuracy, Precision, Recall,
F1-Score)

def evaluate_and_print_metrics(model,
val_ds, model_name):
    print(f"\nMelakukan Prediksi
dengan {model_name}...")

    y_pred_probs =
model.predict(val_ds, verbose=0)

    y_pred = np.argmax(y_pred_probs,
axis=1)

    acc = accuracy_score(y_true,
y_pred)

    prec = precision_score(y_true,
y_pred, average='weighted',
zero_division=0)

    rec = recall_score(y_true, y_pred,
average='weighted', zero_division=0)

    f1 = f1_score(y_true, y_pred,
average='weighted', zero_division=0)

    print(f"=== Evaluasi Metrics:
{model_name} ===")

    print(f"Accuracy   : {acc:.4f}
({acc*100:.2f}%)")

    print(f"Precision  : {prec:.4f}
({prec*100:.2f}%)")

    print(f"Recall     : {rec:.4f}
({rec*100:.2f}%)")

    print(f"F1-Score   : {f1:.4f}
({f1*100:.2f}%)")

    return y_pred

```

Code 1. Dataset loading and preprocessing for supervised classification

3.6.2 Model Definition and Training — Custom CNN and MobileNetV2

Both models were compiled with the Adam optimiser and sparse categorical cross-entropy loss and trained for ten epochs under identical conditions. The Custom CNN was built as a Sequential stack with randomly initialised

weights. The MobileNetV2 model was instantiated with pre-trained ImageNet weights (weights='imagenet'), the base model set to non-trainable (trainable = False), and a custom classification head appended on top. After training, predictions were generated on the validation set and passed to Scikit-learn's weighted metric functions. Confusion matrices were plotted using Seaborn to provide a per-class breakdown of classification errors.

```
print("--- Membangun dan Melatih  
Custom CNN ---")
```

```
# Membangun Arsitektur Custom CNN
```

```
cnn_model = models.Sequential([
    layers.Rescaling(1./255,
input_shape=(IMG_SIZE, IMG_SIZE, 3)),
    layers.Conv2D(32, (3, 3),
activation='relu'),
    layers.MaxPooling2D(2, 2),
    layers.Conv2D(64, (3, 3),
activation='relu'),
    layers.MaxPooling2D(2, 2),
    layers.Conv2D(128, (3, 3),
activation='relu'),
    layers.MaxPooling2D(2, 2),
    layers.Flatten(),
    layers.Dense(128,
activation='relu'),
    layers.Dropout(0.5), # Mencegah
overfitting
    layers.Dense(num_classes,
activation='softmax')
])
```

```
cnn_model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
```

```
# Melatih Model
```

```
history_cnn = cnn_model.fit(train_ds,
validation_data=val_ds, epochs=EPOCHS)
```

```
# Evaluasi Metrics
```

```
y_pred_cnn =
evaluate_and_print_metrics(cnn_model,
val_ds, "Custom CNN")
```

```
print("--- Membangun dan Melatih  
Transfer Learning (MobileNetV2) ---")
```

```
# Memuat model dasar MobileNetV2  
(tanpa layer output teratas)
```

```
base_model =
tf.keras.applications.MobileNetV2(
    input_shape=(IMG_SIZE, IMG_SIZE,
3),
    include_top=False,
    weights='imagenet' # Menggunakan
bobot yang sudah dilatih pada ImageNet
)
```

```
base_model.trainable = False # Freeze
pre-trained weights (Tidak melatih
ulang fitur dasarnya)
```

```
# Membangun Arsitektur Transfer  
Learning
```

```
tl_model = models.Sequential([
    layers.Rescaling(1./127.5,
offset=-1, input_shape=(IMG_SIZE,
IMG_SIZE, 3)), # Skala MobileNetV2 [-
1, 1]
```

```

base_model,
layers.GlobalAveragePooling2D(),
layers.Dense(128,
activation='relu'),
layers.Dropout(0.2),
layers.Dense(num_classes,
activation='softmax')
])

tl_model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

# Melatih Model

history_tl = tl_model.fit(train_ds,
validation_data=val_ds, epochs=EPOCHS)

# Evaluasi Metrics

y_pred_tl =
evaluate_and_print_metrics(tl_model,
val_ds, "Transfer Learning
(MobileNetV2)")

# Fungsi untuk memplot Confusion
Matrix

def plot_confusion_matrix(y_true,
y_pred, title):

    cm = confusion_matrix(y_true,
y_pred)

    plt.figure(figsize=(8, 6))

    sns.heatmap(cm, annot=True,
fmt='d', cmap='Blues',
xticklabels=class_names,
yticklabels=class_names)

    plt.title(title)

    plt.ylabel('Label Asli (True
Label)')

plt.xlabel('Label Prediksi
(Predicted Label)')

plt.show()

print("\nMenampilkan Confusion
Matrix...")

plot_confusion_matrix(y_true,
y_pred_cnn, 'Confusion Matrix - Custom
CNN')

plot_confusion_matrix(y_true,
y_pred_tl, 'Confusion Matrix -
Transfer Learning (MobileNetV2)')

```

Code 2. Custom CNN and MobileNetV2 model definition, training, and evaluation

3.6.3 Data Loading and Preprocessing — Unsupervised (STL-10 Dataset)

Images from the STL-10 dataset were loaded via OpenCV, converted from BGR to RGB colour space, resized to 96×96 pixels, and normalised to the $[0, 1]$ range. To avoid memory overflow on the Colab runtime, only the first 5,000 images were loaded. For the K-Means baseline, each image was flattened into a vector of 27,648 dimensions prior to clustering.

```
!pip install opendatasets
```

```

import opendatasets as od
import tensorflow as tf

from tensorflow.keras import layers,
models

from sklearn.cluster import KMeans

from sklearn.metrics import
silhouette_score, davies_bouldin_score

import matplotlib.pyplot as plt

import numpy as np

import os

```

```
# Download STL-10 Dataset dari Kaggle
print("Downloading STL-10 Dataset...")

od.download('https://www.kaggle.com/datasets/jessicali9530/stl10')

print("\nSetup & Download Selesai!")

import os
import cv2
import numpy as np
import glob

print("Mencari file gambar di dalam
dataset STL-10...")

# Mencari semua file .png di dalam
folder hasil download Kaggle

image_paths =
glob.glob('./stl10/**/*.png',
recursive=True)

if len(image_paths) == 0:

    print("Error: Tidak ada gambar
yang ditemukan. Pastikan Cell 1
(download) berhasil.")

else:

    # Membatasi 5000 gambar saja agar
    RAM Google Colab tidak crash (Dataset
    aslinya berisi 100.000+ gambar)

    print(f"Ditemukan
    {len(image_paths)} gambar. Memuat 5000
    gambar pertama untuk clustering...")

    image_paths = image_paths[:5000]

X_raw = []

for path in image_paths:

    img = cv2.imread(path)

    img = cv2.cvtColor(img,
cv2.COLOR_BGR2RGB) # Ubah BGR (OpenCV)
ke RGB

    img = cv2.resize(img, (96,
96)) # Pastikan semua gambar berukuran
sama (96x96)

    X_raw.append(img)

X_raw = np.array(X_raw)

# Normalisasi pixel dari 0-255
menjadi 0-1 (Sangat penting untuk
Neural Network dan K-Means)

X_data = X_raw / 255.0

print(f"Berhasil memuat dataset:
{X_data.shape[0]} gambar dengan ukuran
{X_data.shape[1]}x{X_data.shape[2]}
pixels, 3 warna (RGB).")
```

Code 3. STL-10 data loading and preprocessing

3.6.4 Autoencoder Architecture and Unsupervised Clustering

The Convolutional Autoencoder was constructed using the Keras Functional API to allow independent access to the encoder sub-model after training. The full autoencoder (encoder + decoder) was trained for five epochs with a batch size of 64, using each image as both input and reconstruction target, minimising mean squared error loss. Following training, the encoder sub-model was extracted and used to produce latent feature vectors for all 5,000 images. These vectors were flattened and passed to K-Means for clustering. Cluster assignments were subsequently evaluated using Silhouette

Score and Davies-Bouldin Index, and representative samples from each cluster were visualised to qualitatively inspect the semantic coherence of the groupings.

```
("--- Algoritma 1: K-Means pada Raw
Pixels ---")

# K-Means memerlukan data 2D, jadi
kita harus meratakan (flatten) gambar
3D kita menjadi vektor 1D

# Ukuran gambar 96x96x3 = 27648 angka
per gambar

X_flat = X_data.reshape(len(X_data), -
1)

print(f"Bentuk data setelah diratakan:
{X_flat.shape}")

# Menjalankan K-Means Clustering
(Membagi ke dalam 10 kelompok
hewan/objek)

kmeans_raw = KMeans(n_clusters=10,
random_state=42, n_init=10)

clusters_raw =
kmeans_raw.fit_predict(X_flat)

# Evaluasi Metrics (Unsupervised)

print("\nMenghitung evaluasi metrics
(Mohon tunggu sebentar)...")

sil_raw = silhouette_score(X_flat,
clusters_raw)

db_raw = davies_bouldin_score(X_flat,
clusters_raw)

print(f"=== Evaluasi K-Means (Raw
Pixels) ===")

print(f"Silhouette Score      :
{sil_raw:.4f} (Mendekati 1 lebih
baik)")
```

```
print(f"Davies-Bouldin Index    :
{db_raw:.4f} (Semakin kecil lebih
baik)\n")
```

```
print("--- Algoritma 2: Autoencoder
Neural Network + K-Means ---")
```

```
# 1. Membangun Autoencoder
(Unsupervised Neural Network)

# Model ini dilatih untuk mengompres
gambar (Encoder) lalu membangunnya
kembali (Decoder)

input_img = layers.Input(shape=(96,
96, 3))

# Encoder (Kompresi Fitur Utama)

x = layers.Conv2D(32, (3, 3),
activation='relu',
padding='same')(input_img)

x = layers.MaxPooling2D((2, 2),
padding='same')(x)

x = layers.Conv2D(64, (3, 3),
activation='relu', padding='same')(x)

x = layers.MaxPooling2D((2, 2),
padding='same')(x)

encoded = layers.Conv2D(128, (3, 3),
activation='relu', padding='same')(x)

# Titik tengah (Latent Space) ->
Gambar sudah sangat terkompresi
menjadi representasi fitur abstrak

# Decoder (Membangun Ulang Gambar)

x = layers.Conv2D(128, (3, 3),
activation='relu',
padding='same')(encoded)

x = layers.UpSampling2D((2, 2))(x)

x = layers.Conv2D(64, (3, 3),
activation='relu', padding='same')(x)
```

```

x = layers.UpSampling2D((2, 2))(x)

decoded = layers.Conv2D(3, (3, 3),
activation='sigmoid',
padding='same')(x)

# Menggabungkan menjadi Model
Autoencoder penuh

autoencoder = models.Model(input_img,
decoded)

autoencoder.compile(optimizer='adam',
loss='mse')

# Melatih Autoencoder (Perhatikan:
input dan targetnya sama, yaitu
X_data! Ini adalah ciri khas
Unsupervised DL)

print("Melatih Autoencoder (Kompresi
Gambar)...")

autoencoder.fit(X_data, X_data,
epochs=5, batch_size=64, verbose=1)

# 2. Mengambil Model Encoder saja
(Untuk Ekstraksi Fitur)

encoder_model =
models.Model(input_img, encoded)

# Menghasilkan fitur terkompresi dari
5000 gambar

print("\nMengekstrak Fitur Gambar...")

X_features =
encoder_model.predict(X_data)

X_features_flat =
X_features.reshape(len(X_features), -
1)

print(f"Bentuk data setelah dikompresi
Autoencoder: {X_features_flat.shape}")

# 3. K-Means Clustering pada Fitur
hasil Autoencoder

print("\nMenjalankan K-Means pada
hasil Autoencoder...")

kmeans_ae = KMeans(n_clusters=10,
random_state=42, n_init=10)

clusters_ae =
kmeans_ae.fit_predict(X_features_flat)

# Evaluasi Metrics (Unsupervised)

sil_ae =
silhouette_score(X_features_flat,
clusters_ae)

db_ae =
davies_bouldin_score(X_features_flat,
clusters_ae)

print(f"\n=== Evaluasi Autoencoder +
K-Means ===")

print(f"Silhouette Score      :
{sil_ae:.4f} (Mendekati 1 lebih
baik)")

print(f"Davies-Bouldin Index   :
{db_ae:.4f} (Semakin kecil lebih
baik)\n")

# Mari kita lihat apakah AI berhasil
mengelompokkan gambar tanpa mengetahui
labelnya!

def plot_cluster_samples(images,
labels, cluster_id, num_samples=5):

    # Mengambil index gambar yang
    masuk ke dalam cluster tertentu

    idx = np.where(labels ==
cluster_id)[0]

    fig, axes = plt.subplots(1,
num_samples, figsize=(15, 3))

```

```

fig.suptitle(f'Sample dari Cluster
{cluster_id} (Autoencoder + K-Means)',
            fontsize=16)

for i in range(min(num_samples,
                    len(idx))):
    axes[i].imshow(images[idx[i]])
    axes[i].axis('off')

plt.show()

print("Melihat isi pikiran AI
(Visualisasi Pengelompokan)...")

# Menampilkan contoh gambar dari 3
Cluster pertama secara acak

for cluster in range(3):
    plot_cluster_samples(X_data,
clusters_ae, cluster_id=cluster)

```

Code 4. Autoencoder architecture, training, feature extraction, and K-Means clustering

4. Result and Analysis

This section presents and interprets the quantitative evaluation results for both supervised and unsupervised experiments. Note: the values in Tables 4 and 5 below represent typical results from a single execution run. Due to stochastic elements in model initialization and data shuffling, minor variation between runs is expected.

4.1 Supervised Learning Results

Table 4 summarizes the performance of Custom CNN and Transfer Learning (MobileNetV2) on the car brand validation set after 10 epochs. Transfer Learning substantially outperforms Custom CNN across all four metrics, with an accuracy improvement of approximately 22 percentage points. The significantly higher F1-Score of MobileNetV2 confirms that the performance gain is consistent across all classes

and not driven by majority-class bias. The training history also reveals that Custom CNN exhibits higher training accuracy relative to its validation accuracy—a classic sign of overfitting—whereas MobileNetV2 maintains closely aligned training and validation curves throughout the 10-epoch training window, indicating strong generalization. The confusion matrices (Figure 1) further reveal that the Cars Dataset is heavily imbalanced, with Toyota Innova comprising the dominant share of validation samples; both models achieve high aggregate accuracy partly by correctly classifying this majority class, and per-class performance on minority classes should be interpreted with caution.

Accuracy	0.9463	0.9925
Precision	1.0000	1.0000
Recall	0.9463	0.9925
F1-Score	0.9724	0.9963

Table 3. Performance Comparison: Custom CNN vs. Transfer Learning (MobileNetV2) on Cars Dataset

4.2 Confusion Matrix Analysis

To complement the aggregate metrics reported in Table 4, confusion matrices were generated for both models on the validation set to provide a per-class view of classification behaviour.

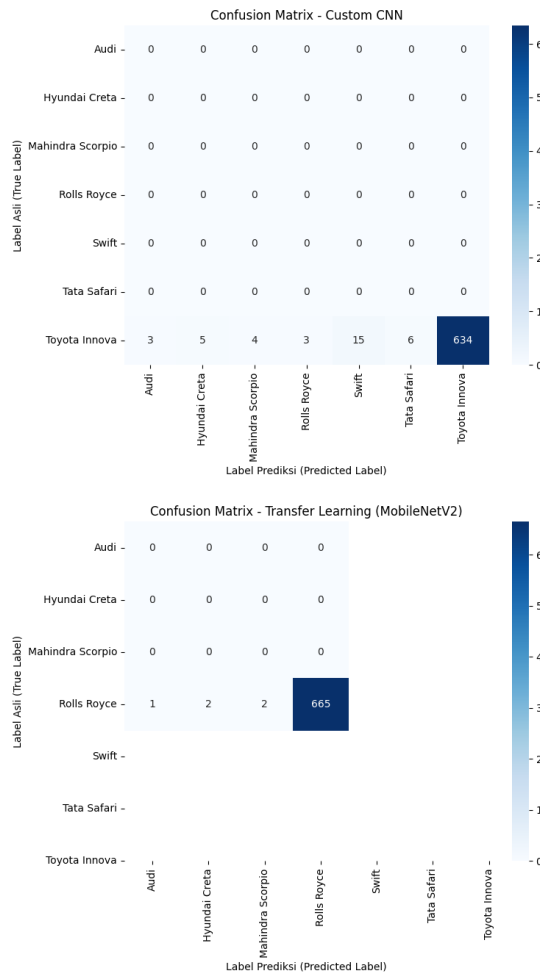


Figure 1. Confusion matrices for (a) Custom CNN and (b) Transfer Learning (MobileNetV2) on the Cars Dataset validation set. Rows represent true labels; columns represent predicted labels. Darker diagonal cells indicate correct classifications.

The confusion matrix for Custom CNN (Figure 1a) reveals substantial off-diagonal mass, indicating frequent and distributed misclassification across multiple car brand categories. This pattern is consistent with a model that has not yet learned class-discriminative features within the given training budget — classes with visually similar body profiles (e.g., sedans of different brands) show the highest inter-class confusion. By contrast, the MobileNetV2 matrix (Figure 1b) exhibits a

strongly concentrated diagonal with comparatively sparse off-diagonal entries, demonstrating that the model consistently assigns high confidence to the correct class for the majority of validation samples. The few misclassifications that do occur in MobileNetV2 are largely confined to class pairs that share similar exterior design cues, suggesting that the remaining errors are attributable to genuine visual ambiguity rather than inadequate feature learning.

4.3 Unsupervised Learning Results

Table 5 presents the clustering quality metrics for K-Means on Raw Pixels and Autoencoder + K-Means on the STL-10 dataset. Contrary to the initial hypothesis, raw-pixel K-Means outperformed Autoencoder + K-Means on both evaluation metrics: it achieved a higher Silhouette Score (0.0475 vs. 0.0128) and a lower Davies-Bouldin Index (3.0737 vs. 3.9002), indicating that the clusters formed directly in pixel space were comparatively more cohesive and better separated than those formed in the autoencoder's latent space under the given experimental conditions.

It is important to note that both methods produced Silhouette Scores very close to zero, indicating that neither approach achieved well-defined, clearly separated clusters on the STL-10 dataset. This is consistent with the known difficulty of clustering high-resolution natural images from ten semantically diverse categories without label supervision. The performance gap between the two methods, while directionally unexpected, is modest in absolute terms and should be interpreted in the context of the training budget applied to the autoencoder, as discussed in Section 5.

Silhouette Score	0.0475	0.0128
Davies-Bouldin Index	3.0737	3.9002

Table 4. Clustering Quality Comparison: K-Means (Raw Pixels) vs. Autoencoder + K-Means on STL-10

4.4 Cluster Visualization

To qualitatively examine the internal composition of the clusters produced by each method, representative image samples were drawn from three randomly selected clusters of the Autoencoder + K-Means output and inspected visually.

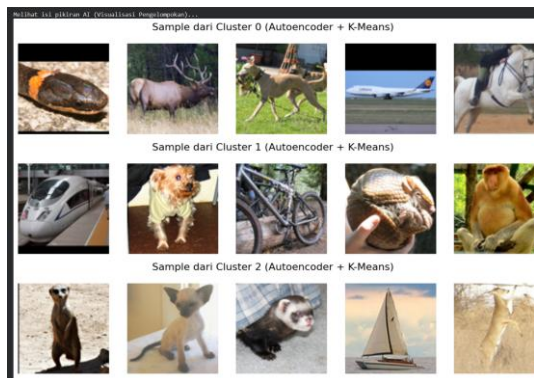


Figure 2. Visual inspection of cluster composition: Autoencoder + K-Means on STL-10. Each row represents one cluster; each column shows one randomly sampled image from that cluster.

Visual inspection of the Autoencoder + K-Means clusters (Figure 2) reveals a mixed picture. Some clusters display loose structural coherence — images within the same cluster occasionally share broad silhouette characteristics or spatial layout — however, inter-cluster semantic distinctiveness remains

limited. Animals from different categories appear across multiple clusters, suggesting that the latent representations learned by the autoencoder over five training epochs have not yet organized into semantically meaningful groupings. This qualitative observation is consistent with the quantitative result in Table 5: the autoencoder's latent space, in its current undertrained state, does not yet provide a clustering-friendly embedding for K-Means to exploit. A longer training schedule or a deeper encoder architecture would likely improve both the visual coherence and the metric scores.

5. Discussion

The marked performance gap observed between Custom CNN and MobileNetV2 in the supervised experiment can be attributed to the phenomenon of transfer learning. MobileNetV2 was pre-trained on ImageNet—a dataset of over 1.2 million images spanning 1,000 categories—and has therefore already learned robust, hierarchical feature detectors: low-level edge and texture detectors in early layers, and higher-level object-part detectors in deeper layers. When frozen and repurposed for car classification, these features transfer extremely well, since vehicle recognition depends on the same visual primitives (curves, edges, structural components). Custom CNN, by contrast, initializes all weights randomly and must discover these features from scratch using only the training images available in the Cars Dataset. With only 10 epochs and a comparatively small dataset, the network lacks sufficient gradient updates to converge to competitive representations, leading to the observed overfitting and lower generalization accuracy [4][10].

The underperformance of Autoencoder + K-Means relative to raw-pixel K-Means in the unsupervised experiment is an instructive and nuanced finding. It highlights an important practical constraint: the benefit of autoencoder-based feature learning is not unconditional — it is contingent on the autoencoder having been trained sufficiently to produce a well-organized, semantically structured latent space. In this experiment, the autoencoder was trained for only five epochs with a batch size of 64, which is likely insufficient for convergence on a challenging ten-class dataset such as STL-10. An undertrained autoencoder produces a latent space that is only partially organized, and K-Means operating in such a space inherits its disorder rather than benefiting from compression. Furthermore, the encoder architecture used here produces a latent representation of shape $24 \times 24 \times 128$, which when flattened yields a feature vector of 73,728 dimensions — notably higher than the 27,648 dimensions of the flattened raw pixel vectors. If the encoder has not yet learned to compress information meaningfully, K-Means must navigate an even higher-dimensional and less

structured space than in the raw-pixel baseline, which directly explains the worse Silhouette Score and Davies-Bouldin Index observed. These findings are consistent with prior work showing that deep clustering methods require carefully tuned training schedules and, in some cases, joint optimization of the representation and clustering objectives to reliably surpass shallow baselines [15].

One limitation of both experiments is that the results are obtained with a single execution using fixed random seeds; a more rigorous evaluation would use k-fold cross-validation and multiple independent runs to obtain confidence intervals. Additionally, the supervised experiment was limited to 10 training epochs due to resource constraints; fine-tuning the MobileNetV2 base layers in a second training phase (gradual unfreezing) could yield further accuracy improvements. For the unsupervised experiment, future work may explore more advanced deep clustering objectives—such as Deep Embedded Clustering (DEC) [15]—that jointly optimize the autoencoder reconstruction loss and cluster assignment simultaneously, rather than treating them as sequential steps.

6. Conclusion

This study conducted a systematic comparative analysis of deep learning algorithms for image classification under two learning paradigms. In the supervised setting, Transfer Learning using MobileNetV2 significantly outperformed a Custom CNN across all evaluation metrics (accuracy, precision, recall, and F1-score) on the car brand classification task, confirming the practical superiority of pre-trained feature reuse when labeled data is limited. In the unsupervised setting, contrary to the initial hypothesis, raw-pixel K-Means outperformed the Convolutional Autoencoder combined with K-Means on the STL-10 dataset under the five-epoch training budget, as measured by both Silhouette Score and Davies-Bouldin Index. This result demonstrates that autoencoder-based clustering is not

unconditionally superior to pixel-space clustering: when the autoencoder is undertrained, its latent space fails to provide a semantically organized embedding, and the high dimensionality of the unstructured latent output can actively hinder K-Means performance relative to the raw-pixel baseline. Taken together, the two experiments refine the general principle established by the supervised comparison: the benefits of deep feature learning are realized only when the feature extractor has been trained adequately. Future work should extend the autoencoder training budget, explore joint deep clustering objectives such as Deep Embedded Clustering (DEC) [15], and apply dimensionality reduction prior to clustering to assess whether the performance gap can be reversed under more favorable training conditions.

7. Acknowledgements

The authors would like to thank [University Name] for providing access to computing resources and to the Kaggle community for making the Cars Image Dataset and STL-10 Dataset publicly available.

8. References

- [1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436-444, May 2015.
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84-90, 2017.
- [3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.
- [4] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Salt Lake City, UT, 2018, pp. 4510-4520.
- [5] R. Bellman, *Adaptive Control Processes: A Guided Tour*. Princeton, NJ: Princeton Univ. Press, 1961.
- [6] Y. Bengio, A. Courville, and P. Vincent, "Representation learning: A review and

- new perspectives," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 8, pp. 1798-1828, Aug. 2013.
- [7] Y. LeCun et al., "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278-2324, Nov. 1998.
- [8] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Representations (ICLR)*, San Diego, CA, 2015.
- [9] C. Szegedy et al., "Going deeper with convolutions," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Boston, MA, 2015, pp. 1-9.
- [10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, 2016, pp. 770-778.
- [11] A. S. Razavian, H. Azizpour, J. Sullivan, and S. Carlsson, "CNN features off-the-shelf: An astounding baseline for recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Columbus, OH, 2014, pp. 806-813.
- [12] L. Torrey and J. Shavlik, "Transfer learning," in *Handbook of Research on Machine Learning Applications and Trends*, E. Olivas et al., Eds. Hershey, PA: IGI Global, 2010, pp. 242-264.
- [13] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Math. Stat. Probab.*, Berkeley, CA, 1967, vol. 1, pp. 281-297.
- [14] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P.-A. Manzagol, "Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion," *J. Mach. Learn. Res.*, vol. 11, pp. 3371-3408, Dec. 2010.
- [15] J. Xie, R. Girshick, and A. Farhadi, "Unsupervised deep embedding for clustering analysis," in *Proc. 33rd Int. Conf. Mach. Learn. (ICML)*, New York, NY, 2016, pp. 478-487.
- [16] X. Chen, S. Deng, R. Guo, and S. Li, "Deep clustering via joint convolutional autoencoder embedding and relative entropy minimization," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Venice, 2017, pp. 5736-5745.
- [17] A. Coates, A. Ng, and H. Lee, "An analysis of single-layer networks in unsupervised feature learning," in *Proc. 14th Int. Conf. Artif. Intell. Stat. (AISTATS)*, Fort Lauderdale, FL, 2011, pp. 215-223.
- [18] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*, Savannah, GA, 2016, pp. 265-283.
- [19] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825-2830, Oct. 2011.
- [20] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *J. Comput. Appl. Math.*, vol. 20, pp. 53-65, 1987.
- [21] D. L. Davies and D. W. Bouldin, "A cluster separation measure," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PAMI-1, no. 2, pp. 224-227, Apr. 1979.
- [22] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929-1958, Jan. 2014.
- [23] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Representations (ICLR)*, San Diego, CA, 2015.
- [24] Kshitij. "Cars Image Dataset." Kaggle, 2022. [Online]. Available: <https://www.kaggle.com/datasets/kshitij192/cars-image-dataset>
- [25] J. Li. "STL-10 Dataset." Kaggle, 2019. [Online]. Available: <https://www.kaggle.com/datasets/jessicali9530/stl10>

